

System Design - Master Notes

A Complete Reference for Interview Preparation, Production Work, Real-World Projects, and System Understanding.

Java Full Stack Master Course

Table of Contents

SYSTEM DESIGN — MASTER NOTES	6
A Complete Reference Book for Interviews, Production Work & System Understanding	6
The Capstone Section — Applying Everything From This Entire Course	6
TOPIC 1: SOLID PRINCIPLES	6
1. Introduction	6
1.1 What is it, and why?	6
2. S — Single Responsibility Principle	6
3. O — Open/Closed Principle	6
4. L — Liskov Substitution Principle	7
5. I — Interface Segregation Principle	7
6. D — Dependency Inversion Principle	7
7. Real Life Analogy	8
8. Frequently Asked Interview Questions	8
TOPIC 2: DESIGN PATTERNS	8
1. Introduction	8
2. Creational Patterns	9
Singleton (Recap: Core Java's Constructors Topic)	9
Factory	9
Builder (Recap: Core Java's Constructors Topic's Mention of the Builder Pattern)	9
3. Structural Patterns	10
Decorator (Recap: Spring's AOP Conceptually Achieves a Similar Effect)	10
4. Behavioral Patterns	10
Strategy (Recap: Directly How Spring Autowires Multiple PaymentProcessor Implementations)	10
Observer (Recap: Directly the Concept Behind Kafka's Pub/Sub and Redis Pub/Sub Sections)	10
5. Real Life Analogy	10
6. Frequently Asked Interview Questions	10
TOPIC 3: LOW-LEVEL DESIGN BASICS	11
1. Introduction	11

2. The LLD Process — A Worked Mini-Example (Parking Lot System)	11
3. Real Life Analogy	12
4. Frequently Asked Interview Questions	12

TOPIC 4: HIGH-LEVEL DESIGN BASICS **12**

1. Introduction	12
2. A Typical HLD Diagram Structure	12
3. HLD vs LLD — Comparison Table	12
4. Real Life Analogy	13
5. Frequently Asked Interview Questions	13

TOPIC 5: SCALABILITY **13**

1. Introduction	13
2. Vertical vs Horizontal Scaling	13
3. Real Life Analogy	13
4. Frequently Asked Interview Questions	13

TOPIC 6: LOAD BALANCER (SYSTEM DESIGN PERSPECTIVE) **14**

1. Introduction	14
2. Load Balancing Algorithms	14
3. Real Life Analogy	14
4. Frequently Asked Interview Questions	14

TOPIC 7: CACHING (SYSTEM DESIGN PERSPECTIVE) **15**

1. Introduction	15
2. Cache Eviction Policies — What Happens When the Cache Is Full	15
3. Where Caching Can Happen (Multiple Layers)	15
4. Real Life Analogy	15
5. Frequently Asked Interview Questions	15

TOPIC 8: DATABASE SCALING **15**

1. Introduction	16
2. Read Replicas	16

3. Sharding — Horizontal Database Partitioning	16
4. Real Life Analogy	16
5. Frequently Asked Interview Questions	16

TOPIC 9: CAP THEOREM 17

1. Introduction	17
2. The Three Properties	17
3. Why You Must Choose — Partition Tolerance Is (Practically) Mandatory	17
4. Real Life Analogy	17
5. Frequently Asked Interview Questions	17

TOPIC 10: CONSISTENT HASHING 18

1. Introduction	18
2. The Problem With Naive Hashing	18
3. The Consistent Hashing Solution	18
4. Real Life Analogy	18
5. Frequently Asked Interview Questions	19

TOPIC 11: RATE LIMITING 19

1. Introduction	19
2. Common Rate Limiting Algorithms	19
3. Real Life Analogy	20
4. Frequently Asked Interview Questions	20

TOPIC 12: MESSAGE QUEUES 20

1. Introduction	20
2. Why Message Queues Matter in System Design — Recap and Framing	20
3. Real Life Analogy	20
4. Frequently Asked Interview Questions	21

TOPIC 13: CDN (CONTENT DELIVERY NETWORK) 21

1. Introduction	21
2. How It Works	21

3. Real Life Analogy	21
4. Frequently Asked Interview Questions	21

TOPIC 14: URL SHORTENER DESIGN (FULL WORKED EXAMPLE) 22

1. Requirements	22
2. API Design (Recap: REST APIs Section)	22
3. Data Model (Recap: PostgreSQL/Hibernate Sections)	22
4. Core Design Decision — Generating the Short Code	22
5. High-Level Architecture	22
6. Key Trade-offs (Interview-Critical Discussion Points)	22
7. Frequently Asked Interview Questions	23

TOPIC 15: CHAT SYSTEM DESIGN (FULL WORKED EXAMPLE) 23

1. Requirements	23
2. Core Design Decision — WebSockets, Not Plain REST	23
3. High-Level Architecture	24
4. The Hard Problem — Routing a Message to the RIGHT Server	24
5. Key Trade-offs	24
6. Frequently Asked Interview Questions	24

TOPIC 16: NOTIFICATION SYSTEM DESIGN (FULL WORKED EXAMPLE) 25

1. Requirements	25
2. High-Level Architecture — Bringing Together Nearly Every Section of This Course	25
3. Why an Event-Driven Design (Recap: Microservices Section) Is Essential Here	25
4. Handling Failures — Retry and Dead Letter Queues	25
5. Preventing Duplicate Notifications — Idempotency	26
6. Key Trade-offs	26
7. Frequently Asked Interview Questions	26
8. Revision Notes (Entire System Design Section Consolidated)	27
9. Homework (Covers Entire System Design Section)	27
FINAL SUMMARY — SYSTEM DESIGN COMPLETE	27

SYSTEM DESIGN — MASTER NOTES

A Complete Reference Book for Interviews, Production Work & System Understanding

The Capstone Section — Applying Everything From This Entire Course

TOPIC 1: SOLID PRINCIPLES

1. Introduction

1.1 What is it, and why?

SOLID is an ACRONYM for five OBJECT-ORIENTED DESIGN PRINCIPLES (coined by Robert C. Martin, "Uncle Bob") that, followed together, produce code that's easier to MAINTAIN, EXTEND, and TEST — directly building on Core Java's OOP topics (Inheritance, Polymorphism, Abstraction, Encapsulation), providing concrete GUIDANCE on HOW to actually apply those pillars well in real design decisions.

2. S — Single Responsibility Principle

A class should have ONLY ONE reason to change — exactly the principle repeated throughout this course (Core Java's Methods topic, Advanced Java's MVC topic).

```
// VIOLATES SRP - this class has MULTIPLE reasons to change (business logic AND persistence AND notification)
class Employee {
    void calculateBonus() { /* business logic */ }
    void saveToDatabase() { /* persistence logic */ } // a DB schema change forces THIS class to change too!
    void sendWelcomeEmail() { /* notification logic */ } // an email provider change ALSO forces a change here!
}

// FOLLOWS SRP - each responsibility lives in its OWN class
class Employee { /* just data + core business logic */ }
class EmployeeRepository { void save(Employee e) { /* ... */ } } // persistence - separate reason to change
class NotificationService { void sendWelcomeEmail(Employee e) { /* ... */ } } // notification - separate reason
```

3. O — Open/Closed Principle

Classes should be OPEN for extension, but CLOSED for modification — you should be able to ADD new behavior WITHOUT changing EXISTING, already-tested code.

```
// VIOLATES OCP - adding a NEW payment type means MODIFYING this existing method
double calculateFee(String paymentType) {
    if (paymentType.equals("CREDIT_CARD")) return 2.5;
    else if (paymentType.equals("PAYPAL")) return 3.0;
    // adding "CRYPTO" means EDITING this method, risking breaking already-tested logic!
}
```

```
// FOLLOWS OCP - a NEW payment type is added by CREATING a new class, never touching existing ones
interface PaymentMethod { double calculateFee(); }
class CreditCardPayment implements PaymentMethod { public double calculateFee() { return 2.5; } }
class PayPalPayment implements PaymentMethod { public double calculateFee() { return 3.0; } }
class CryptoPayment implements PaymentMethod { public double calculateFee() { return 1.0; } } // just
ADD this!
```

4. L — Liskov Substitution Principle

A subclass must be substitutable for its parent class WITHOUT breaking correctness — an object of a subtype should behave consistently with what callers of the SUPERTYPE expect (recap: Core Java's Inheritance topic mentioned this exact principle).

```
// VIOLATES LSP - the classic example
class Rectangle {
protected int width, height;
void setWidth(int w) { width = w; }
void setHeight(int h) { height = h; }
int getArea() { return width * height; }
}
class Square extends Rectangle {
@Override void setWidth(int w) { width = w; height = w; } // BREAKS the parent's expected behavior!
@Override void setHeight(int h) { width = h; height = h; } // a caller expecting Rectangle's contract
} // (independent width/height) gets SURPRISED

// A caller relying on Rectangle's contract breaks silently when given a Square:
void resizeAndCheck(Rectangle r) {
r.setWidth(5);
r.setHeight(10);
assert r.getArea() == 50; // FAILS if 'r' is actually a Square! (area would be 100)
}
```

5. I — Interface Segregation Principle

Clients should not be forced to depend on methods they don't use — prefer several small, focused interfaces over one large, "fat" interface (recap: Spring Core's AOP topic mentioned focused interfaces).

```
// VIOLATES ISP - a "fat" interface forces EVERY implementer to provide EVERYTHING, even irrelevant
methods
interface Worker {
void work();
void eat(); // a ROBOT worker doesn't eat - forced to implement a meaningless method!
}

// FOLLOWS ISP - segregated into focused, specific interfaces
interface Workable { void work(); }
interface Eatable { void eat(); }
class HumanWorker implements Workable, Eatable { /* implements BOTH, genuinely needs both */ }
class RobotWorker implements Workable { /* implements ONLY what it actually needs */ }
```

6. D — Dependency Inversion Principle

Depend on ABSTRACTIONS, not concrete implementations — this is EXACTLY the IoC/Dependency Injection principle from Spring Core's Topic 1, restated as a formal design principle.

```
// VIOLATES DIP - directly depends on a CONCRETE class (recap: Spring Core's IoC problem statement!)
class OrderService {
private MySQLDatabase database = new MySQLDatabase(); // tightly coupled to ONE specific
```

```

implementation
}

// FOLLOWS DIP - depends on an ABSTRACTION (interface), the concrete implementation is INJECTED
class OrderService {
private final Database database; // depends on the INTERFACE, not a specific DB
OrderService(Database database) { this.database = database; } // injected - EXACTLY Spring's DI (Core
Topic 2)
}

```

7. Real Life Analogy

- **Restaurant:** SOLID is like a restaurant's design philosophy — each STATION has ONE clear job (SRP), NEW dishes are added without rewriting EXISTING recipes (OCP), any SUBSTITUTE chef at a station must perform CONSISTENTLY with what that station's role requires (LSP), staff aren't forced to learn irrelevant skills outside their actual role (ISP), and the KITCHEN'S overall workflow depends on GENERAL roles ("a cook," "a server"), not one SPECIFIC named individual (DIP) — so any properly-trained substitute can step in seamlessly.

8. Frequently Asked Interview Questions

- 1 **What does SOLID stand for?** Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion.
- 2 **What is the Single Responsibility Principle?** A class should have only ONE reason to change — one clear, focused responsibility.
- 3 **What is the Open/Closed Principle?** Classes should be open for EXTENSION (adding new behavior) but closed for MODIFICATION (not requiring changes to existing, tested code).
- 4 **How does the Dependency Inversion Principle relate to Spring's IoC/DI?** They're the SAME core idea — depend on abstractions (interfaces) rather than concrete implementations, with the actual implementation supplied externally (injected), rather than created internally.
- 5 **What's a classic example violating the Liskov Substitution Principle?** Making `Square` extends `Rectangle` and overriding `setWidth/setHeight` to keep both dimensions equal — this breaks code written against `Rectangle`'s expected contract of independently-settable width/height.

TOPIC 2: DESIGN PATTERNS

1. Introduction

What is it? Design Patterns are well-established, PROVEN, NAMED solutions to RECURRING software design problems — a shared VOCABULARY letting developers communicate complex design ideas quickly ("just use a Factory here" instead of explaining the whole approach from scratch every time).

2. Creational Patterns

Singleton (Recap: Core Java's Constructors Topic)

```
public class ConfigManager {
    private static final ConfigManager INSTANCE = new ConfigManager();
    private ConfigManager() { } // private constructor - recap Core Java!
    public static ConfigManager getInstance() { return INSTANCE; }
}
```

Factory

```
interface NotificationSender { void send(String message); }
class EmailSender implements NotificationSender { public void send(String m) { /* ... */ } }
class SmsSender implements NotificationSender { public void send(String m) { /* ... */ } }

class NotificationFactory {
    static NotificationSender create(String type) {
        return switch (type) {
            case "EMAIL" -> new EmailSender();
            case "SMS" -> new SmsSender();
            default -> throw new IllegalArgumentException("Unknown type");
        };
    }
}
```

Builder (Recap: Core Java's Constructors Topic's Mention of the Builder Pattern)

```
public class Employee {
    private final String name;
    private final double salary;
    private final String department;

    private Employee(Builder b) { this.name = b.name; this.salary = b.salary; this.department = b.department; }

    public static class Builder {
        private String name;
        private double salary;
        private String department;
        public Builder name(String n) { this.name = n; return this; }
        public Builder salary(double s) { this.salary = s; return this; }
        public Builder department(String d) { this.department = d; return this; }
        public Employee build() { return new Employee(this); }
    }
}

Employee emp = new Employee.Builder().name("Alice").salary(75000).department("Engineering").build();
```

3. Structural Patterns

Decorator (Recap: Spring's AOP Conceptually Achieves a Similar Effect)

```
interface Coffee { double cost(); }
class SimpleCoffee implements Coffee { public double cost() { return 2.0; } }

class MilkDecorator implements Coffee {
    private final Coffee coffee;
    MilkDecorator(Coffee coffee) { this.coffee = coffee; }
    public double cost() { return coffee.cost() + 0.5; } // ADDS behavior, without modifying SimpleCoffee!
}

Coffee order = new MilkDecorator(new SimpleCoffee());
System.out.println(order.cost()); // 2.5
```

4. Behavioral Patterns

Strategy (Recap: Directly How Spring Autowires Multiple PaymentProcessor Implementations)

```
interface DiscountStrategy { double apply(double price); }
class NoDiscount implements DiscountStrategy { public double apply(double p) { return p; } }
class TenPercentOff implements DiscountStrategy { public double apply(double p) { return p * 0.9; } }

class Checkout {
    private final DiscountStrategy strategy;
    Checkout(DiscountStrategy strategy) { this.strategy = strategy; } // SAME idea as Spring's constructor injection!
    double finalPrice(double price) { return strategy.apply(price); }
}
```

Observer (Recap: Directly the Concept Behind Kafka's Pub/Sub and Redis Pub/Sub Sections)

```
interface Observer { void update(String event); }
class EmailNotifier implements Observer { public void update(String e) { System.out.println("Email: " + e); } }

class EventPublisher {
    private final List<Observer> observers = new ArrayList<>();
    void subscribe(Observer o) { observers.add(o); }
    void publish(String event) { observers.forEach(o -> o.update(event)); } // notify ALL subscribers
}
```

5. Real Life Analogy

- **Restaurant:** Design Patterns are like a chef's collection of STANDARD, NAMED cooking TECHNIQUES ("braising," "sautéing") — any trained chef immediately understands EXACTLY what's meant by the name, without needing the full technique re-explained from scratch every single time, the same way "use a Singleton" instantly communicates a specific, well-understood design approach to another developer.

6. Frequently Asked Interview Questions

- 1 **What is a design pattern?** A well-established, proven, named solution to a recurring software design problem, providing a shared vocabulary for developers.

- 2 **What is the Singleton pattern, and where has it appeared earlier in this course?** Ensuring only ONE instance of a class exists, via a private constructor and a static access method — used in Core Java's Constructors topic and conceptually in Spring's default `singleton` bean scope.
 - 3 **What is the Strategy pattern, and how does it relate to Spring's Dependency Injection?** Defining a family of interchangeable algorithms/behaviors behind a common interface — exactly the pattern Spring uses when multiple `@Component` implementations of an interface are autowired and swapped via configuration.
 - 4 **What is the Observer pattern, and where has it appeared earlier in this course?** Objects ("observers") subscribing to be notified of events from a "publisher" — the exact conceptual foundation of Kafka's Pub/Sub and Redis Pub/Sub, covered in earlier sections.
-
-

TOPIC 3: LOW-LEVEL DESIGN BASICS

1. Introduction

What is it? Low-Level Design (LLD) is designing the ACTUAL CLASSES, their RELATIONSHIPS, and their METHODS for a specific system component — essentially, applying EVERY Core Java OOP concept from this course (classes, interfaces, inheritance, SOLID) to model a REAL problem before writing implementation code.

2. The LLD Process — A Worked Mini-Example (Parking Lot System)

```
1. IDENTIFY the core ENTITIES (nouns): ParkingLot, ParkingSpot, Vehicle, Ticket
2. IDENTIFY their RELATIONSHIPS: a ParkingLot HAS-MANY ParkingSpots; a ParkingSpot
   HOLDS ONE Vehicle at a time
3. IDENTIFY BEHAVIORS (verbs): parkVehicle(), removeVehicle(), findAvailableSpot()
4. APPLY OOP/SOLID: use an ABSTRACT ParkingSpot class with subtypes (CompactSpot,
   LargeSpot) for different vehicle sizes (recap: Core Java's Inheritance)
```

```
enum VehicleType { COMPACT, LARGE }
```

```
abstract class ParkingSpot {
    private final String spotId;
    private Vehicle currentVehicle;
    boolean isAvailable() { return currentVehicle == null; }
    void parkVehicle(Vehicle v) { this.currentVehicle = v; }
    abstract boolean canFit(VehicleType type); // POLYMORPHISM - each spot type decides differently
}
```

```
class CompactSpot extends ParkingSpot {
    boolean canFit(VehicleType type) { return type == VehicleType.COMPACT; }
}
```

```
class ParkingLot {
    private final List<ParkingSpot> spots;
    Optional<ParkingSpot> findAvailableSpot(VehicleType type) {
        return spots.stream()
            .filter(ParkingSpot::isAvailable)
            .filter(spot -> spot.canFit(type))
    }
}
```

```
.findFirst();  
}  
}
```

3. Real Life Analogy

- **College:** LLD is like designing the DETAILED BLUEPRINT for one specific building on campus (exact room layouts, door placements, wiring) — a precise, detailed plan for ONE component, as opposed to the campus-wide master plan (High-Level Design, next topic) showing how ALL buildings relate to each other overall.

4. Frequently Asked Interview Questions

- 1 **What is Low-Level Design?** Designing the actual classes, their relationships, and methods for a specific system component, applying OOP principles and SOLID design.
- 2 **What's a common first step in an LLD interview question?** Identifying the core ENTITIES (nouns) in the problem, their relationships, and their behaviors, before writing any actual code.

TOPIC 4: HIGH-LEVEL DESIGN BASICS

1. Introduction

What is it? High-Level Design (HLD) is designing the OVERALL ARCHITECTURE of a system — which major COMPONENTS/SERVICES exist, how they COMMUNICATE, and how DATA flows between them — directly applying the Microservices section's concepts (API Gateway, Service Discovery, Event-Driven Architecture) at the whole-system level, rather than any single class's internal details.

2. A Typical HLD Diagram Structure

```
[Client] -> [Load Balancer] -> [API Gateway] -> [Microservices] -> [Databases]  
|  
[Cache Layer (Redis)]  
|  
[Message Queue (Kafka)]
```

3. HLD vs LLD — Comparison Table

Aspect	High-Level Design	Low-Level Design
Scope	ENTIRE system architecture	ONE specific component/module
Concerns	Scalability, data flow, service boundaries	Classes, methods, OOP relationships
Typical output	Architecture/box diagrams	Class diagrams, actual code structure
Recap from this course	Microservices section, AWS section	Core Java OOP, SOLID (Topic 1)

4. Real Life Analogy

- **College:** HLD is like the CAMPUS-WIDE master plan — where the library, dorms, and cafeteria are located RELATIVE to each other, and how students/resources FLOW between them — the big picture, before zooming into any ONE building's detailed blueprint (LLD).

5. Frequently Asked Interview Questions

- 1 **What is High-Level Design?** Designing the overall system architecture — major components, their communication, and data flow — as opposed to the detailed internals of any single component.
- 2 **How does HLD relate to the Microservices section covered earlier?** HLD is essentially APPLYING those concepts (API Gateway, service boundaries, event-driven communication) to design a specific system's overall architecture.

TOPIC 5: SCALABILITY

1. Introduction

What is it? Scalability is a system's ability to handle INCREASED load (more users, more data, more requests) by adding RESOURCES — the CENTRAL concern of virtually every system design interview question.

2. Vertical vs Horizontal Scaling

VERTICAL SCALING ("scale UP"): add MORE POWER to a SINGLE server
(more CPU, more RAM) - recap: choosing a LARGER EC2 instance type (AWS section)
LIMIT: there's a PHYSICAL ceiling to how big ONE machine can get

HORIZONTAL SCALING ("scale OUT"): add MORE SERVERS/instances
- recap: AWS Auto Scaling Groups + Load Balancer (AWS section)
ADVANTAGE: virtually UNLIMITED scaling, plus BETTER fault tolerance
(one server failing doesn't take down the WHOLE system)

3. Real Life Analogy

- **Restaurant:** Vertical scaling is like making ONE kitchen BIGGER and adding more equipment to it — eventually you run out of physical space. Horizontal scaling is like opening MULTIPLE separate kitchen locations, each handling a SHARE of the total demand — a fundamentally more scalable, more resilient approach (recap: exactly why Auto Scaling + Load Balancer, AWS section, work together).

4. Frequently Asked Interview Questions

- 1 **What's the difference between vertical and horizontal scaling?** Vertical scaling adds more resources to a SINGLE server; horizontal scaling adds MORE servers/instances, distributing load across them.

- 2 **Why is horizontal scaling generally preferred for large-scale systems?** It has no practical upper limit (unlike vertical scaling's physical hardware ceiling) and improves fault tolerance, since no single server failure takes down the entire system.
-

TOPIC 6: LOAD BALANCER (SYSTEM DESIGN PERSPECTIVE)

1. Introduction

(The AWS section's Load Balancer topic covered PRACTICAL setup — this topic covers the ALGORITHMIC/CONCEPTUAL side relevant to system design INTERVIEWS.)

2. Load Balancing Algorithms

Algorithm	How It Works
Round Robin	Requests distributed to servers in simple ROTATING order
Least Connections	Routes to whichever server currently has the FEWEST active connections
IP Hash	Routes based on a hash of the CLIENT'S IP — the SAME client consistently reaches the SAME server (useful for maintaining session affinity)
Weighted Round Robin	Servers with MORE capacity receive a PROPORTIONALLY larger share of requests

3. Real Life Analogy

- **Restaurant:** Round Robin is like a host seating customers at tables in strict, predictable ROTATION. Least Connections is like seating the NEXT customer at whichever currently-open table's waiter has the FEWEST other tables to attend to right now.

4. Frequently Asked Interview Questions

- 1 **Name three common load balancing algorithms.** Round Robin, Least Connections, IP Hash.
 - 2 **When would IP Hash be preferred over Round Robin?** When you need the SAME client to consistently reach the SAME backend server (session affinity), rather than being routed unpredictably each time.
-

TOPIC 7: CACHING (SYSTEM DESIGN PERSPECTIVE)

1. Introduction

(Full Redis-specific caching mechanics were covered in the Redis section's Caching topic — this topic covers cache **EVICTON POLICIES** and **PLACEMENT STRATEGIES** relevant to system design interviews.)

2. Cache Eviction Policies — What Happens When the Cache Is Full

Policy	How It Decides What to Remove
LRU (Least Recently Used)	Removes the item that hasn't been ACCESSED in the LONGEST time — the most common default
LFU (Least Frequently Used)	Removes the item with the FEWEST total accesses
FIFO (First In, First Out)	Removes the OLDEST -inserted item, regardless of access patterns

3. Where Caching Can Happen (Multiple Layers)

Client Browser Cache -> CDN (Topic 11) -> Application Cache (Redis) -> Database Query Cache

4. Real Life Analogy

- **Restaurant:** LRU is like a small countertop cache only keeping the ingredients used **MOST RECENTLY**, discarding whatever hasn't been touched in the longest time when space runs low — a natural, effective strategy since **RECENTLY**-used ingredients are statistically likely to be needed again soon.

5. Frequently Asked Interview Questions

- 1 **What does LRU eviction mean?** When the cache is full, the item that hasn't been **ACCESSED** in the longest time is removed to make room for new entries.
- 2 **Name the multiple layers at which caching can happen in a real system.** Browser cache, CDN, application-level cache (Redis), and database query cache.

TOPIC 8: DATABASE SCALING

1. Introduction

What is it? As a system grows, a SINGLE database server eventually can't handle the load alone — Database Scaling covers the STANDARD techniques (beyond RDS's basic managed hosting, AWS section) for distributing database load across multiple servers.

2. Read Replicas

```
WRITES go here
|
v
[PRIMARY Database]
/ | \
v v v
[Replica 1] [Replica 2] [Replica 3] <- READS distributed across these
(recap: RDS supports read replicas natively!)
```

Since most applications are READ-HEAVY (many more reads than writes), replicating the database and directing READ traffic to replicas dramatically increases overall read throughput, while all WRITES still go to the single primary (to maintain consistency).

3. Sharding — Horizontal Database Partitioning

Instead of ONE giant database holding ALL data, SPLIT it across MULTIPLE databases ("shards"), each holding a SUBSET (e.g., users A-M on Shard 1, users N-Z on Shard 2) - directly enabling HORIZONTAL scaling (Topic 5) for the DATABASE layer itself, not just the application servers.

4. Real Life Analogy

- **Bank:** Read Replicas are like a bank maintaining SEVERAL identical copies of its account records specifically for CUSTOMERS CHECKING their balance (reads), while ALL actual money-moving TRANSACTIONS (writes) still go through ONE authoritative central ledger — keeping the single source of truth consistent while spreading out the read load. Sharding is like the bank splitting customer records ACROSS different regional branches based on customer ID range, so no SINGLE branch needs to hold the ENTIRE bank's data.

5. Frequently Asked Interview Questions

- 1 **What is a read replica, and why is it useful?** A copy of the primary database that serves READ queries, offloading read traffic from the primary (which continues handling writes) — useful since most applications are read-heavy.
 - 2 **What is database sharding?** Horizontally partitioning a database across multiple servers, each holding a SUBSET of the total data, enabling the database layer itself to scale beyond what one server can handle.
-
-

TOPIC 9: CAP THEOREM

1. Introduction

What is it? CAP Theorem states that a DISTRIBUTED system can only GUARANTEE TWO of the following THREE properties SIMULTANEOUSLY, during a network failure: **Consistency, Availability, Partition Tolerance** — a fundamental, unavoidable trade-off in distributed systems design.

2. The Three Properties

Property	Meaning
Consistency (C)	Every read receives the MOST RECENT write, or an error — all nodes see the SAME data at the SAME time
Availability (A)	Every request receives a RESPONSE (not necessarily the most up-to-date one), even if some nodes have failed
Partition Tolerance (P)	The system CONTINUES operating despite network failures splitting it into disconnected groups ("partitions")

3. Why You Must Choose — Partition Tolerance Is (Practically) Mandatory

In ANY real distributed system, network partitions (temporary connectivity failures BETWEEN servers) WILL eventually happen - so Partition Tolerance is NOT really optional in practice. This means the REAL choice is between:

CP (Consistency + Partition Tolerance): during a partition, the system REFUSES some requests to guarantee data stays consistent (sacrifices Availability) - e.g., traditional relational databases prioritizing correctness, like a bank's core ledger

AP (Availability + Partition Tolerance): during a partition, the system KEEPS responding to requests, even if some responses might be slightly STALE/inconsistent temporarily (sacrifices strict Consistency) - e.g., many NoSQL databases, or a social media "like count" that can be briefly inconsistent across servers

4. Real Life Analogy

- **Bank:** A CP choice is like a bank REFUSING to process a withdrawal request during a system outage between branches, rather than risk allowing a withdrawal that might create an INCONSISTENT balance across branches — correctness over availability. An AP choice is like a social media platform's "like" counter STILL letting you like a post during a partial outage, even if the DISPLAYED count might be briefly, slightly out of sync across different users — availability over perfect real-time consistency.

5. Frequently Asked Interview Questions

- 1 **What does CAP Theorem state?** A distributed system can only guarantee two of Consistency, Availability, and Partition Tolerance simultaneously during a network partition.
 - 2 **Why is Partition Tolerance considered practically mandatory rather than a genuine choice?** Network partitions are an unavoidable REALITY in any real distributed system — the actual design decision is really between prioritizing Consistency (CP) or Availability (AP) when a partition occurs.
 - 3 **Give a real-world example of a system that would prioritize Consistency (CP) over Availability.** A banking/financial ledger system, where showing a temporarily stale or incorrect balance is worse than briefly refusing a request.
-

TOPIC 10: CONSISTENT HASHING

1. Introduction

What is it? Consistent Hashing is an algorithm for DISTRIBUTING data across MULTIPLE servers (e.g., a distributed Redis cache cluster) in a way that MINIMIZES data movement when servers are ADDED or REMOVED — solving a genuine problem with NAIVE hashing approaches.

2. The Problem With Naive Hashing

NAIVE approach: $\text{server} = \text{hash}(\text{key}) \% \text{NUMBER_OF_SERVERS}$

Adding or removing EVEN ONE server changes NUMBER_OF_SERVERS, which COMPLETELY changes the result of $\text{hash}(\text{key}) \% \text{NUMBER_OF_SERVERS}$ for NEARLY EVERY key - meaning ALMOST ALL cached data suddenly maps to a DIFFERENT server, causing a MASSIVE, unnecessary cache-invalidation event just from ONE server being added/removed!

3. The Consistent Hashing Solution

Imagine a CIRCULAR ring of hash values (0 to some MAX value). BOTH servers AND keys are hashed onto THIS SAME ring. A key is assigned to the NEXT server found CLOCKWISE from its position.

```
Server A
/ \
Key3 Server B
| |
Key1 Key2
\ /
Server C
```

Adding a NEW server ONLY affects the keys BETWEEN it and the NEXT server clockwise - ALL OTHER keys remain assigned to their ORIGINAL servers, completely UNAFFECTED - a DRAMATIC improvement over naive hashing's "everything reshuffles" problem.

4. Real Life Analogy

- **Restaurant:** Naive hashing is like a restaurant chain that, the MOMENT it opens ONE additional location, has to COMPLETELY REASSIGN every single existing customer to a possibly totally different location based on a formula that just happened to change. Consistent Hashing is like assigning customers based on their POSITION along a continuous, circular map — opening ONE new location only affects the customers geographically NEAREST to that new spot, leaving everyone else's assignment completely undisturbed.

5. Frequently Asked Interview Questions

- 1 **What problem does Consistent Hashing solve?** It minimizes the amount of data that needs to be REMAPPED when servers are added or removed from a distributed system, unlike naive `hash(key) % N` approaches where nearly ALL keys get reassigned.
- 2 **How does Consistent Hashing conceptually work?** Both servers and keys are placed on a CONCEPTUAL circular ring (via hashing); each key is assigned to the next server found going clockwise — adding/removing a server only affects the keys in its immediate vicinity on the ring.
- 3 **Where would Consistent Hashing be genuinely useful, based on earlier sections of this course?** Distributing keys across a CLUSTER of Redis cache servers, or across Kafka partitions/brokers, without triggering a massive data-remapping event whenever the cluster's server count changes.

TOPIC 11: RATE LIMITING

1. Introduction

What is it? Rate Limiting restricts HOW MANY requests a client can make within a given TIME WINDOW — protecting a system from abuse, accidental overload, or a single client monopolizing shared resources. (A *basic implementation using Redis's INCR+EXPIRE was already shown in the Redis section's Expiration topic — this topic covers the underlying ALGORITHMS.*)

2. Common Rate Limiting Algorithms

Algorithm	How It Works
Fixed Window	Count requests within a FIXED time block (e.g., "100 requests per calendar minute") — simple, but allows a BURST right at the window boundary (close to 200 requests in a brief moment spanning two windows)
Sliding Window	Tracks requests within a CONTINUOUSLY MOVING window (the last 60 seconds, RELATIVE to NOW) — more accurate, avoids the fixed-window boundary burst issue
Token Bucket	A "bucket" holds a limited number of TOKENS, refilled at a steady rate; each request CONSUMES one token — allows brief BURSTS (using saved-up tokens) while enforcing a steady AVERAGE rate over time

3. Real Life Analogy

- **Bank:** Fixed Window rate limiting is like a bank strictly allowing "100 withdrawals per CALENDAR hour" — a customer could withdraw 100 times at 11:59 and ANOTHER 100 times at 12:01, technically within the rules but functionally 200 withdrawals in 2 minutes. Token Bucket is like a bank issuing a customer a REFILLING allowance of transaction "tokens" at a steady rate, letting them SAVE UP a few for occasional bursts of activity, while still enforcing a steady LONG-TERM average limit.

4. Frequently Asked Interview Questions

- 1 **What problem does the "Fixed Window" rate limiting algorithm have?** It allows a burst of nearly DOUBLE the intended limit right at the boundary between two consecutive windows.
- 2 **What is the Token Bucket algorithm?** Tokens refill a "bucket" at a steady rate; each request consumes a token, allowing brief bursts (using saved tokens) while enforcing a steady average rate over time.
- 3 **How was basic rate limiting implemented in the Redis section?** Using `INCR` to count requests and `EXPIRE` to set the counting window's time-to-live.

TOPIC 12: MESSAGE QUEUES

1. Introduction

(Kafka, covered in its own dedicated section, IS a message queue/streaming system — this topic frames message queues CONCEPTUALLY, at the system design level, for interview purposes.)

2. Why Message Queues Matter in System Design — Recap and Framing

A message queue DECOUPLES a PRODUCER (that generates work) from a CONSUMER (that processes it) - EXACTLY the Event-Driven Architecture topic from the Microservices section, and EXACTLY what Kafka provides.

Real system design use cases:

- SMOOTHING OUT traffic spikes: a sudden burst of orders queues up instead of overwhelming the order-processing service directly
- DECOUPLING services: an "OrderPlaced" event can trigger payment, inventory, AND notification services INDEPENDENTLY, without the Order service needing direct, synchronous knowledge of any of them

3. Real Life Analogy

- **Restaurant:** A message queue is like a kitchen's order-ticket rail — the FRONT-OF-HOUSE (producer) can take orders as fast as customers arrive, WITHOUT needing to wait for the kitchen (consumer) to finish each dish before taking the NEXT order — the rail absorbs the difference in PACE between the two, exactly as covered in the Kafka section.

4. Frequently Asked Interview Questions

- 1 **What core problem does a message queue solve in a system design context?** Decoupling producers from consumers, and smoothing out traffic spikes by absorbing bursts of work into a queue rather than overwhelming a downstream service directly.
 - 2 **Which section of this course covered a real, production-grade message queue/streaming platform in depth?** The Kafka section.
-

TOPIC 13: CDN (CONTENT DELIVERY NETWORK)

1. Introduction

What is it? A CDN is a GEOGRAPHICALLY DISTRIBUTED network of servers that cache and serve CONTENT (images, videos, your React frontend's static files) from a location PHYSICALLY CLOSE to each user, dramatically reducing latency compared to every user fetching content from ONE central origin server.

2. How It Works

User in Tokyo -> nearest CDN edge server (Tokyo) -> (if cached) INSTANT response
-> (if NOT cached) fetches from the ORIGIN server ONCE, then CACHES it for future Tokyo requests

User in London -> nearest CDN edge server (London) -> SAME process, independently

3. Real Life Analogy

- **Restaurant:** A CDN is like a restaurant CHAIN opening LOCAL branches close to customers in every major city, instead of requiring EVERY customer worldwide to travel to ONE single flagship location — each local branch stocks the SAME popular menu items (cached content), served much faster to NEARBY customers than shipping everything from one central kitchen every time.

4. Frequently Asked Interview Questions

- 1 **What is a CDN?** A geographically distributed network of servers caching and serving content from locations physically close to users, reducing latency.
 - 2 **What kind of content is typically served via a CDN?** Static assets — images, videos, CSS/JavaScript bundles (like your compiled React application from the React section).
-
-

TOPIC 14: URL SHORTENER DESIGN (FULL WORKED EXAMPLE)

1. Requirements

- **Functional:** Given a long URL, generate a SHORT code (e.g., `short.ly/aZ3xQ`); given a short code, REDIRECT to the original long URL.
- **Non-functional:** Extremely READ-HEAVY (many more redirects than URL creations), LOW LATENCY redirects, high AVAILABILITY.

2. API Design (Recap: REST APIs Section)

```
POST /api/urls { "longUrl": "https://very-long-url.com/..." } -> { "shortCode": "aZ3xQ" }
GET /aZ3xQ -> HTTP 302 Redirect to the original long URL
```

3. Data Model (Recap: PostgreSQL/Hibernate Sections)

```
CREATE TABLE urls (
  short_code VARCHAR(10) PRIMARY KEY,
  long_url TEXT NOT NULL,
  created_at TIMESTAMP DEFAULT NOW(),
  click_count BIGINT DEFAULT 0
);
```

4. Core Design Decision — Generating the Short Code

```
// APPROACH: Base62 encoding of an auto-incrementing database ID (recap: Core Java's number systems!)
// Base62 uses [a-zA-Z0-9] - 62 characters - producing SHORT, URL-safe codes from a numeric ID

long id = 125_000_000L; // an auto-incrementing ID from the database
String shortCode = Base62.encode(id); // e.g., "8M0kX" - MUCH shorter than the raw number, URL-safe
```

Using an auto-incrementing ID + Base62 encoding GUARANTEES uniqueness (no collision-checking needed, unlike random-string generation) and keeps codes SHORT even for a huge number of URLs.

5. High-Level Architecture

```
[Client] -> [Load Balancer] -> [API Gateway] -> [URL Service]
|
+-----+-----+
| |
[Redis Cache] [PostgreSQL/RDS]
(hot short codes, (source of truth,
recap: Redis Caching read replicas
section!) for scale)
```

6. Key Trade-offs (Interview-Critical Discussion Points)

- **Why cache aggressively (Redis)?** The system is EXTREMELY read-heavy (redirects vastly outnumber creations) — exactly the cache-aside pattern from the Redis section.

- **Why NOT worry much about CAP theorem's Consistency here?** A short-lived, SLIGHT delay in a NEWLY created URL being available everywhere is acceptable (AP-leaning) — unlike a banking system, a moment's inconsistency causes no real harm.
- **Scaling reads:** Read replicas (Topic 8) + Redis caching handle the read-heavy load; the URL Service itself scales horizontally (Topic 5) behind the Load Balancer.

7. Frequently Asked Interview Questions

- 1 **Why is Base62 encoding of an auto-incrementing ID a good approach for generating short codes?** It guarantees uniqueness without needing collision detection, and produces consistently short, URL-safe codes even at massive scale.
- 2 **Why should this system heavily favor caching?** It's extremely read-heavy — redirects vastly outnumber new URL creations, making Redis caching (cache-aside pattern) highly effective.
- 3 **Where does this system lean on the CAP theorem spectrum, and why?** Availability-leaning (AP) — a brief delay in full consistency for a newly-created URL is an acceptable trade-off for high availability and low latency on redirects.

TOPIC 15: CHAT SYSTEM DESIGN (FULL WORKED EXAMPLE)

1. Requirements

- **Functional:** Users send/receive messages in REAL TIME; message history is persisted and retrievable.
- **Non-functional:** LOW LATENCY delivery, support MANY concurrent connections, messages must not be LOST.

2. Core Design Decision — WebSockets, Not Plain REST

Recap the JavaScript section's Event Loop and the Fetch API: plain HTTP request/response (client asks, server answers) doesn't naturally support the SERVER pushing a NEW message to a client the INSTANT it arrives.

WEBSOCKETS solve this: a PERSISTENT, BIDIRECTIONAL connection between client and server, allowing the SERVER to push data to the client IMMEDIATELY, without the client needing to repeatedly ask "any new messages yet?" (a wasteful technique called "polling").

3. High-Level Architecture

```
[User A] <--WebSocket--> [Chat Server 1] <---\
\
[User B] <--WebSocket--> [Chat Server 2] <----- [Kafka: "new-message" topic] (recap: Kafka section!)
/
[User C] <--WebSocket--> [Chat Server 3] <---/
|
v
[PostgreSQL: message history]
[Redis: "who's currently online / connected to WHICH server"]
```

4. The Hard Problem — Routing a Message to the RIGHT Server

User A (connected to Chat Server 1) sends a message to User B (connected to a DIFFERENT instance, Chat Server 2).

SOLUTION: Chat Server 1 PUBLISHES the message to a Kafka topic (recap: Kafka's Consumer Groups topic - EVERY chat server instance subscribes and receives it), and Chat Server 2 (which holds User B's actual live WebSocket connection) delivers it to User B directly.

This EXACTLY mirrors the Event-Driven Architecture pattern from the Microservices section - servers don't need DIRECT knowledge of each other's connections; Kafka decouples them.

5. Key Trade-offs

- **Why Kafka over a direct server-to-server call?** Chat servers scale horizontally (Topic 5) and instances come/go dynamically (recap: Service Discovery, Microservices section) — Kafka decouples "which specific server holds User B's connection" from "which server received User A's message."
- **CAP theorem lean:** Message DELIVERY favors Availability (a brief delay is tolerable), but message HISTORY/persistence favors Consistency (you don't want to LOSE a sent message) — different parts of the SAME system can make DIFFERENT CAP trade-offs.
- **Redis's role:** tracking WHICH server currently holds each user's live connection (a fast, frequently-updated lookup — exactly Redis's strength from its own section).

6. Frequently Asked Interview Questions

- 1 **Why use WebSockets instead of plain REST/HTTP polling for a chat system?** WebSockets provide a persistent, bidirectional connection allowing the server to PUSH new messages instantly, avoiding the latency and wasted resources of the client repeatedly polling for new messages.
- 2 **How does a message get from a user on one server to a recipient connected to a DIFFERENT server instance?** Via a message queue (like Kafka) — the sending server publishes the message, and ALL chat server instances (each a consumer) receive it, with the specific instance holding the recipient's actual connection delivering it directly.
- 3 **Can different parts of the same system make different CAP theorem trade-offs?** Yes — message delivery might favor Availability (a brief delay is fine), while message history/persistence favors Consistency (a sent message must never be lost) — CAP trade-offs are made PER COMPONENT, not necessarily uniformly across an entire system.

TOPIC 16: NOTIFICATION SYSTEM DESIGN (FULL WORKED EXAMPLE)

1. Requirements

- **Functional:** Send notifications to users via MULTIPLE channels (email, SMS, push notification) triggered by various application EVENTS (order placed, password reset).
- **Non-functional:** Must NOT block the triggering application's main request flow; must handle FAILURES gracefully (a failed SMS provider shouldn't lose the notification).

2. High-Level Architecture — Bringing Together Nearly Every Section of This Course

```
[Any Microservice] -- publishes "SendNotification" event --> [Kafka Topic]
|
+-----+-----+-----+-----+-----+-----+
| | |
[Email Consumer Group] [SMS Consumer Group] [Push Consumer Group]
(recap: Kafka Consumer (recap: Kafka Consumer (recap: Kafka Consumer
Groups topic - each Groups - INDEPENDENT Groups - INDEPENDENT
TYPE is its OWN group) from the others) from the others)
| | |
[Spring Boot Email Service [SMS Provider API [Push Notification
- JavaMailSender, call - via Feign Client, Service API call]
Spring Boot section!] Microservices section!]
```

3. Why an Event-Driven Design (Recap: Microservices Section) Is Essential Here

BAD (synchronous): OrderService directly calls EmailService.sendEmail() and WAITS for it to complete before returning a response to the user - a SLOW or DOWN email provider directly slows down/breaks ORDER PLACEMENT itself! (recap: the exact Resilience4j cascading-failure problem)

GOOD (event-driven): OrderService just PUBLISHES an event and returns IMMEDIATELY - notification delivery happens INDEPENDENTLY, and a temporarily-down email provider has ZERO impact on order placement.

4. Handling Failures — Retry and Dead Letter Queues

If the Email Consumer fails to send (provider temporarily down):

1. RETRY with exponential backoff (wait 1s, then 2s, then 4s, ...)
2. After MAX retries, move the message to a "Dead Letter Queue" (DLQ) - a SEPARATE topic holding "notifications that repeatedly failed" - for later manual investigation/reprocessing, rather than SILENTLY losing the notification entirely

5. Preventing Duplicate Notifications — Idempotency

Recap: the REST APIs section's note on POST not being idempotent, and Kafka's "at-least-once" delivery guarantees (a message MIGHT be processed more than once during a consumer rebalance/retry).

SOLUTION: each notification event carries a UNIQUE ID; the consumer checks (via Redis, recap: Redis section) "have I ALREADY sent this exact notification ID?" before sending - preventing a user from getting the SAME email twice due to a retry.

6. Key Trade-offs

- **Event-driven over synchronous:** decouples notification delivery from the triggering business operation entirely (recap: Microservices Event-Driven Architecture topic).
- **Separate consumer groups per channel:** Email/SMS/Push failures are ISOLATED from each other — an SMS provider outage doesn't affect email delivery at all (recap: Kafka's independent consumer groups).
- **Idempotency via Redis:** prevents duplicate sends from Kafka's at-least-once delivery semantics.

7. Frequently Asked Interview Questions

- 1 **Why should a notification system be built using event-driven architecture rather than direct synchronous calls?** So a slow/failing notification provider (email, SMS) never blocks or breaks the CORE business operation (like order placement) that triggered it — the triggering service simply publishes an event and moves on immediately.
- 2 **What is a Dead Letter Queue, and why is it useful?** A separate queue holding messages that repeatedly failed processing after retries, preventing silent data loss and enabling later manual investigation/reprocessing.
- 3 **How do you prevent a user from receiving the same notification twice, given Kafka's at-least-once delivery guarantee?** Track a unique ID per notification event and check (e.g., via Redis) whether it's already been processed before sending, making the consumer's processing effectively idempotent.
- 4 **Why use SEPARATE consumer groups for email, SMS, and push notifications?** So a failure or slowdown in ONE channel (e.g., the SMS provider being down) doesn't affect or delay the OTHER channels at all — each processes independently.

8. Revision Notes (Entire System Design Section Consolidated)

SYSTEM DESIGN — CHEAT SHEET

=====

SOLID: Single Responsibility, Open/Closed, Liskov Substitution, Interface Segregation, Dependency Inversion (= Spring's IoC/DI principle, formalized)

DESIGN PATTERNS: Singleton/Factory/Builder (creational), Decorator (structural), Strategy/Observer (behavioral) - a shared design vocabulary

LLD: design CLASSES/relationships for ONE component (applies Core Java OOP)

HLD: design the OVERALL architecture (applies Microservices/AWS concepts)

SCALABILITY: vertical (bigger server, has a ceiling) vs horizontal (more servers, ~unlimited + fault tolerant) - horizontal generally preferred

LOAD BALANCER algorithms: Round Robin, Least Connections, IP Hash

CACHING: LRU/LFU/FIFO eviction; layers: browser -> CDN -> app cache -> DB cache

DATABASE SCALING: read replicas (read-heavy load) + sharding (horizontal partitioning of the DATA itself)

CAP THEOREM: pick 2 of Consistency/Availability/Partition Tolerance -

Partition Tolerance is practically mandatory, so the REAL choice is

CP (bank ledgers) vs AP (social media like counts)

CONSISTENT HASHING: ring-based key/server hashing - adding/removing a server only remaps NEARBY keys, not everything (unlike naive hash % N)

RATE LIMITING: Fixed Window (simple, boundary-burst flaw) / Sliding Window (accurate) / Token Bucket (allows bursts, steady average)

MESSAGE QUEUES: decouple producer/consumer, smooth traffic spikes (= Kafka)

CDN: geographically distributed content caching, close to users

3 WORKED EXAMPLES (apply EVERYTHING from this course):

URL SHORTENER: Base62(auto-increment ID), heavy Redis caching, AP-leaning

CHAT SYSTEM: WebSockets (not polling), Kafka routes messages across server instances, Redis tracks who's connected where

NOTIFICATION SYSTEM: event-driven (never blocks the triggering operation), separate consumer groups per channel, retry + DLQ, Redis-based idempotency against Kafka's at-least-once delivery

9. Homework (Covers Entire System Design Section)

- 1 Refactor a class violating multiple SOLID principles (write one deliberately first) into a properly SOLID-compliant design.
- 2 Design the LLD for a simple "Library Management System" (Book, Member, Loan classes) applying inheritance/polymorphism appropriately.
- 3 Draw the HLD for a simplified e-commerce system, labeling where Load Balancer, API Gateway, Cache, Message Queue, and CDN would each fit.
- 4 Write a one-page design document for the URL Shortener, Chat System, OR Notification System, explicitly stating your CAP theorem trade-off decision and justifying it.

FINAL SUMMARY — SYSTEM DESIGN COMPLETE

This concludes the **System Design** section — 16 topics, and with it, the **ENTIRE Java Full Stack Master Course**:

- 1 **SOLID Principles** — the formal design rules underlying good OOP, directly connected to Spring's IoC/DI.
- 2 **Design Patterns** — Singleton, Factory, Builder, Decorator, Strategy, Observer.
- 3 **Low-Level Design** — applying Core Java OOP to model real components.
- 4 **High-Level Design** — applying Microservices/AWS concepts to whole-system architecture.
- 5 **Scalability** — vertical vs horizontal.
- 6 **Load Balancer** — algorithms, building on the AWS section.
- 7 **Caching** — eviction policies, building on the Redis section.
- 8 **Database Scaling** — read replicas, sharding.
- 9 **CAP Theorem** — the fundamental distributed-systems trade-off.
- 10 **Consistent Hashing** — minimizing remapping on cluster changes.
- 11 **Rate Limiting** — Fixed Window, Sliding Window, Token Bucket.
- 12 **Message Queues** — decoupling producers/consumers, building on Kafka.
- 13 **CDN** — geographically distributed content delivery.
- 14 **URL Shortener Design** — a full worked example.
- 15 **Chat System Design** — a full worked example.
- 16 **Notification System Design** — a full worked example.

Congratulations — you have now completed the entire Java Full Stack Master Course, from `public static void main(String[] args)` all the way through designing and reasoning about large-scale, distributed production systems. Every concept in this final section directly reused and connected back to something built earlier in the course — Core Java's OOP and SOLID, Spring's DI, Redis's caching, Kafka's messaging, AWS's infrastructure, and Docker's deployment model — because that is genuinely how real-world system design works: composing the fundamentals you've now mastered.

(END OF SYSTEM DESIGN MASTER NOTES — END OF THE JAVA FULL STACK MASTER COURSE.)